Software Requirements Specification

for

CyberID – Cybersecurity Intrusion Detection System

Version: 1.0

Prepared by

N Sukhitha Bhushan	**1RVU23CSE293**	**nsukhithab.btech23@rvu.edu.in**
Swathi Kulkarni	**1RVU23CSE492**	**swathik.btech23@rvu.edu.in**
R Keerthana Prasad	**1RVU23CSE363**	**rkeerthanap.btech23@rvu.edu.in**
Jaee Priyadarshan Kulkarni	**1RVU23CSE196**	**priyadarshank.btech23@rvu.edu.in**

Instructor: ***CVSN Reddy***

Course: **Agile Software Engineering and DevOps**

Lab Section: ***B***

Teaching Assistant: ***N/A***

Date: **20/04/2025**

Contents

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
1.0	N Sukhitha Bhushan	Initial draft version of the Software Requirements Specification (SRS) document. Awaiting review and feedback for further refinement.	20/04/2025

1 Introduction

This project aims to develop a machine learning-based Intrusion Detection System (IDS) that accurately classifies network traffic as either normal or abnormal. By leveraging Support Vector Machine (SVM) algorithm with a Radial Basis Function (RBF) and effective data preprocessing techniques, the system provides an automated, real-time solution for detecting potential security threats in network logs. The core objective is to enhance cybersecurity by reducing manual effort and increasing the reliability of threat detection. The system supports the upload of log files via a web interface and delivers immediate classification results, along with visual insights and explanations for any detected anomalies. In future iterations, the model's performance can be further improved through fine-tuning and the integration of additional classification methods.

1.1 Document Purpose

The purpose of this document is to capture the product requirements for the above project.

- This document will be used by the testers for writing the test plan, test cases, test scripts, test traceability matrix, and test automation.
- This document will be used by the project manager and scrum master for planning and scheduling.
- This document will be used by the DevOps engineer for provisioning and configuring the necessary DevOps tools.
- This document will be used by the product manager and customer for verifying whether the requirements have been accurately captured.

1.2 Product Scope

The Intrusion Detection System is a lightweight, AI-powered application designed to detect and classify abnormal network behavior using machine learning. The system focuses on enhancing cybersecurity monitoring through real-time log analysis and visual feedback. Its core functionalities include:

- Log File Input: Users can upload network log files (e.g., CSV format) through a simple web interface for analysis.
- ML-Based Threat Detection: A trained Support Vector Machine (SVM) model processes the input to classify activity as normal or abnormal.
- Explainable Predictions: The system provides human-readable justifications for each detection, helping users understand flagged anomalies.
- Result Visualization: Threats and patterns are displayed in a clear and intuitive format, aiding in quick interpretation and response.

1.3 Intended Audience and Document Overview

This document is intended for the following key personnel:

1. Developers
2. Testers
3. Product Manager
4. DevOps Engineer
5. Project Manager & Scrum Master
6. Customer

1.4 Definitions, Acronyms and Abbreviations

S.No	*Acronym*	*Expansion*
1	*IDS*	*Intrusion Detection System*
2	*SVM*	*Support Vector Machine*
3	*RBF*	*Radial Basis Function*
4	*ML*	*Machine Learning*
5	*GUI*	*Graphical User Interface*
6	*DL*	*Deep Learning*

1.5 Document Conventions

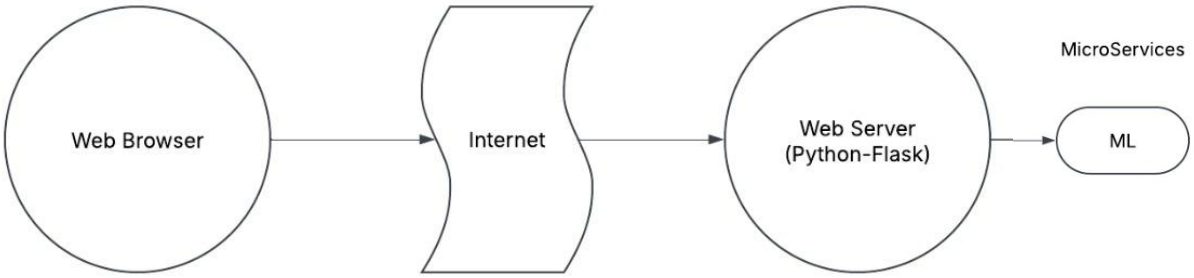
Not Applicable

1.6 References and Acknowledgments

1. Problem statement and dataset provided by the project supervisor
2. Scikit-learn documentation for SVM with RBF kernel
3. Tutorials from Towards Data Science and Analytics Vidhya on intrusion detection
4. Resources on outlier handling and data scaling in cybersecurity
5. Documentation for Flask, Pandas, and Matplotlib
6. GitHub and Stack Overflow for troubleshooting and deployment

2 Overall Description

2.1 Product Overview



This product follows a three-layer architecture. The layers in the product architecture are as follows:

1. GUI – The frontend for user interaction.
2. Web Server Layer - Built with Python Flask, this layer handles user requests and manages communication between the GUI and the ML layer.
3. ML Layer – Contains a Machine Learning model (SVM with RBF kernel) for classifying network activity as normal or an intrusion.

Users upload log files in CSV format. The system applies data preprocessing, including scaling using a pre-fitted scaler, then performs predictions using the trained SVM model. The results are displayed on the GUI for easy interpretation.

2.2 Product Functionality

This product consists of three layers: Web Client, Web Server, and ML Module.

1. Web Client: Allows users to input network log data via CSV file uploads.
2. Web Server: The server handles requests from the web client. The client can request prediction accuracy, and the web server communicates with the backend ML module to provide the prediction results and accuracy.
3. ML Module: This module processes the CSV file input and calculates the prediction accuracy using the SVM model with RBF kernel for intrusion detection.

2.3 Design and Implementation Constraints

- **COMET Methodology**
  The software design will follow the COMET method to ensure modular and structured development.
- **UML Model**
  Unified Model Language (UML) will be used for designing system architecture.
- **Hardware Limitations**
  The system's performance relies on CPU resources for processing data and training the SVM model. The speed of prediction may vary depending on the hardware specifications.
- **Storage Constraints**
  Initially, data will be stored in CSV files, with potential future expansion to a more robust database solution.
- **Security Considerations**
  Secure API communication and data encryption will be implemented to ensure the protection of user data and privacy.
- **Technology Stack**
  The system is built using Python (Flask for backend), with Scikit-learn for SVM modeling. Frontend interaction is supported by basic HTML/CSS with minimal JavaScript, ensuring simplicity for users.

2.4 Assumptions and Dependencies

- Users provide clean, preprocessed network log datasets in .csv format.
- Dependencies are managed using requirements.txt or environment.yml for consistent environment setup.
- No internet access is required for local execution, except for remote model training or updates, if applicable

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system will be accessible via a web browser with the following front-end elements:

File Upload Panel

- Accepts `.csv` files containing network log data.
- Mandatory columns: `src_ip`, `dest_ip`, `protocol`, `src_port`, `dest_port`, `timestamp`, `payload_size`, `flag`, `attack_type`.

Model Selection Menu

- Users can run predictions using the SVM model with an RBF kernel.

Run & Reset Buttons

- Run button triggers model training and prediction.

Results Display Panel

- Shows evaluation metrics: **accuracy, precision, recall, F1-score**.
- Displays **confusion matrix** and **ROC curve**.
- The system visualizes threats and patterns in a clear and intuitive manner, enabling quick interpretation and timely response.

3.1.2 Hardware Interfaces

- No specialized hardware required.
- The system runs efficiently on standard CPU configurations.

3.1.3 Software Interfaces

Frontend → Backend

- *Transfers user configuration and uploaded CSV files as JSON payloads.*

· ***Backend → ML Engine***

- *Invokes Scikit-learn modules for preprocessing and SVM model execution.*
- *Standard function calls and data exchange within the Python environment.*

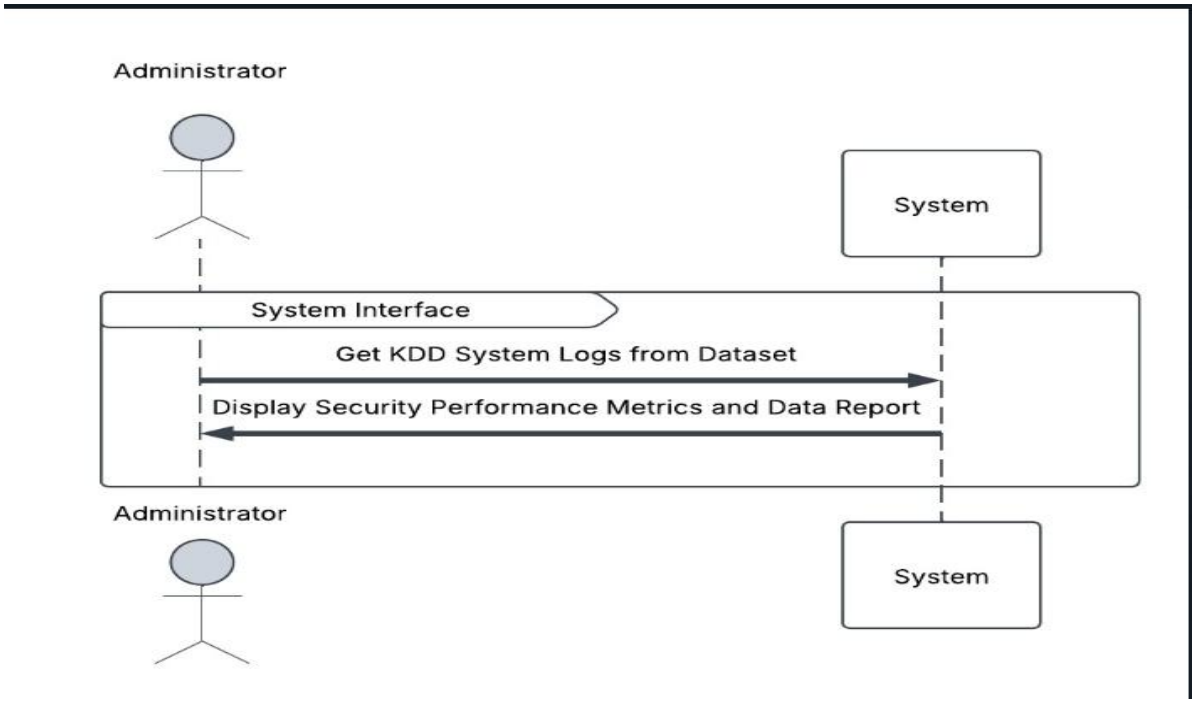
· ***Backend → SQLite Database***

- *Stores session data, uploaded file logs, predictions, and evaluation results with timestamps.*
- *Auto-cleanup after session timeout or user logout*

3.2 Functional Requirements

ID	Functional Requirement Description
FR1	The system shall allow the user to upload CSV datasets containing network log data.
FR2	The system shall validate the dataset structure and format, ensuring mandatory columns such as src_ip, dest_ip, protocol, src_port, dest_port, timestamp, payload_size, flag, and attack_type.
FR3	The system shall train the selected SVM model using the provided dataset.
FR4	The system shall preprocess the data, including normalization, handling missing values, and scaling using a pre-fitted scaler.
FR5	The system shall evaluate the model using cross-validation or a held-out test set for validation.
FR6	The system shall display the following evaluation metrics: accuracy, precision, recall, and F1-score.
FR7	The system shall generate and render a confusion matrix and ROC curve to visualize model performance.
FR8	The system shall store all runs in a local SQLite database, including session metadata, file logs, and model output with timestamps.
FR9	The system shall generate visual analytics such as bar charts, line graphs, and heatmaps to represent log patterns.

3.3 Use Case Model



3.3.1 Use Case #1 (U1 - Upload CSV Dataset)

Purpose - Allows the user to upload a CSV dataset containing system logs for threat detection.

Requirements Traceability –

FR1 The system shall allow the user to upload CSV datasets containing network log data.

FR2 The system shall validate the dataset structure and format, ensuring mandatory columns such as src_ip, dest_ip, protocol, src_port, dest_port, timestamp, payload_size, flag, and attack_type.

Priority - High

Preconditions –

- The user is on the upload page.
- The user has a properly formatted .csv file.

Post conditions –

- File is validated and accepted for processing.
- Errors are shown for invalid format or structure.

Actors -

- **Primary Actor:** User
- **System:** Web application handling the upload and processing.

Extends - None

Flow of Events –

1. Basic Flow

1. The user navigates to the Upload CSV Dataset page.
2. The user clicks the Upload CSV button.
3. The system prompts the user to select a file.
4. The user selects a valid CSV file.
5. The system validates the file format and data structure.
6. If validation is successful, the system uploads and stores the data.
7. The system confirms a successful upload.

2. Alternative Flows

A1: Invalid File Format

- If the user selects a non-CSV file, the system rejects the file and displays an error message**.**

A2: Large File Size

- If the uploaded file exceeds the maximum size, the system displays an error and stops the upload.

A3: Data Format Issues

- If missing or incorrect data is detected, the system Add support for different delimiters (e.g., semicolon, tab).

3. Exceptions

- **E1: System Error** – If the system crashes during upload, it notifies the user and logs the error.
- **E2: Network Failure** – If the user loses connection, the system pauses the upload and allows a retry.

Includes -

- **U2: Validate Data** – Ensures correct file structure before storing.
- **U3: Store Data** – Saves the dataset after validation.

Notes/Issues –

- Define max file size limit.
- Consider implementing progress tracking for large files.

3.3.2 Use Case #2 (U2 - Analyze Network Log Data)

Purpose
Allows the user to analyze uploaded system log data for trends, traffic patterns, and potential anomalies without performing prediction or classification.

Requirements Traceability

FR4: The system shall preprocess the data (normalize, handle nulls).

FR7: The system shall display metrics: accuracy, precision, recall, F1-score (if applicable for labeled data).

FR9: The system shall generate visual analytics such as bar charts, line graphs, and heatmaps to represent log patterns.

Priority
High

Preconditions

- A valid CSV dataset has been uploaded and stored.
- The user is on the Data Analytics dashboard.

Postconditions

- Key statistics and visual analytics are displayed.
- Insights on traffic volume, protocol usage, port scans, or frequent source IPs are shown.
- User can interpret the trends to enhance security posture.

Actors

- **Primary Actor**: User
- **System**: Web application, analytics engine (e.g., pandas/matplotlib/seaborn)

Extends

- Extends: **U1 – Upload CSV Dataset**

Flow of Events

1. **Basic Flow**
    1. The user navigates to the Data Analytics section.
    2. The system loads previously uploaded CSV log data.
    3. The system preprocesses the data (e.g., cleans nulls, formats timestamps).

4. The system generates and displays visual analytics.
5. The user can interact with or download the visual reports.

2. **Alternative Flows**

A1: Unsupported Data Fields

- If a required field for the selected analysis is missing, the system alerts the user.

A2: Large Data Volume

- For large datasets, the system may paginate or summarize the output.

3. **Exceptions**

E1: Visualization Error

- If chart generation fails, the system notifies the user and logs the error.

E2: Session Timeout

- If the session expires before the analysis completes, the user is prompted to reload.

Includes

- **U4: Preprocess Data** – Cleans and normalizes log entries before analysis.
- **U5: Generate Visuals** – Creates bar graphs, heatmaps, or time series charts based on log data.

Notes/Issues

- Consider adding filters (e.g., date range, IP address, port) for focused analysis.
- Optionally, allow exporting visuals as PNG or PDF.

3.3.3 Use Case #3 (U3 - Predict Threats Using ML Model)

Purpose
Allows the user to run the uploaded system log dataset through a pre-trained Machine Learning model to classify and detect abnormal or malicious activity.

Requirements Traceability

- **FR3**: The system shall allow the user to choose between ML or DL models.
- **FR5**: The system shall train the selected model using the dataset (or load a pre-trained one).
- **FR6**: The system shall evaluate the model using cross-validation or a test set.
- **FR7**: The system shall display metrics: accuracy, precision, recall, F1-score.
- **FR8**: The system shall generate a confusion matrix and ROC curve.
- **FR12**: The system shall display model configuration details and time taken.

Priority
High

Preconditions

- The user has uploaded a valid dataset.
- The model has been selected or is pre-configured for inference.

Postconditions

- Predictions are displayed with performance metrics and visualizations.
- The user understands whether the logs represent normal or abnormal behavior.
- Option to export results is provided.

Actors

- **Primary Actor**: User
- **System**: Web application, ML engine (e.g., SVM)

Extends

- **U1 – Upload CSV Dataset**
- **U2 – Analyze Network Log Data**

Flow of Events

1. **Basic Flow**
    1. The user navigates to the Prediction section.
    2. The user selects a machine learning model (e.g., SVM, Random Forest).
    3. The system loads the uploaded dataset and applies preprocessing.
    4. The system applies the selected model to the dataset.
    5. The system displays predicted labels (e.g., normal or abnormal).

6. The system shows evaluation metrics such as accuracy, precision, recall, and F1-score.
7. The system generates a confusion matrix, ROC curve, and summary of model parameters.

2. **Alternative Flows**

A1: No Model Selected

- If the user does not select a model, the system prompts for model selection.

A2: Model Loading Error

- If the model file (.pkl) is missing or corrupted, the system displays an error message.

A3: Input Format Mismatch

- If the dataset has extra or missing features, the system alerts the user.

3. **Exceptions**

E1: Model Execution Timeout

- If the model takes too long to process, the system times out and logs the issue.

E2: Backend Failure

- If the ML engine crashes, the system displays an error and prevents further execution.

Includes

- **U4: Preprocess Data** – Ensures input format matches training features.
- **U6: Display Prediction Results** – Presents threat classification and performance evaluation.

Notes/Issues

- Add option to view individual prediction results (e.g., row-wise normal/attack).
- Consider highlighting top contributing features for explainability (e.g., SHAP values).

4 Other Non-functional Requirements

4.1 Performance Requirements

- **NFR1**: The system shall respond with model evaluation results (including metric scores and visualizations) within **10 seconds** for datasets containing **≤500 records** on a standard CPU-based machine.
- **NFR2**: The system shall support **concurrent usage** by at least **10 users** without performance degradation when hosted on a local server or cloud instance with ≥4 vCPUs and 8 GB RAM.
- **NFR3**: Training and testing operations shall use **efficient batch processing**, and deep learning models shall be optimized with early stopping to minimize CPU/GPU cycles.

4.2 Safety and Security Requirements

- **NFR5**: The system shall implement **user authentication** with username-password login for session-based access, preventing unauthorized use.
- **NFR6**: Uploaded CSV files and session data shall be stored in a temporary directory and shall be automatically deleted after **30 minutes** of inactivity.
- **NFR7**: Inputs will be validated against a strict schema before processing to mitigate injection attacks, malformed data, or corruption of internal models.
- **NFR8**: REST APIs used for model evaluation will be secured via **token-based authentication**, and CSRF protection will be implemented for web form submissions.

4.3 Software Quality Attributes

4.3.1.1 Usability

- ***NFR9****: The interface shall provide tooltips, inline documentation, and validation messages to guide users throughout the process.*
- ***NFR10****: The system shall be accessible via any modern browser (Chrome, Firefox, Edge) with no additional plugins required.*

4.3.1.2 Maintainability

- ***NFR11****: All components (UI, Flask backend, model logic) shall be modularized in separate files and follow standard naming conventions and docstrings.*
- ***NFR12****: The project will include unit tests for each module, and any new feature shall be integrated through Git with code review.*

4.3.1.3 Portability

- ***NFR13****: The system shall be containerized using* ***Docker*** *for platform-independent deployment.*
- ***NFR14****: The software shall be executable on Linux, Windows, and macOS with minimal configuration via a provided installation script or environment file.*

4.3.1.4 Reliability & Availability

- ***NFR15**: The system shall maintain **>99% uptime** when deployed on cloud infrastructure and automatically log runtime errors for developer review.*
- ***NFR16**: In the event of a backend failure, the system shall provide a meaningful fallback message and log the incident for investigation.*

5 Other Requirements

5.1 Extensibility and Future Enhancements

- ***OR1**: The system architecture shall support **plug-and-play model integration** for additional cyber threat categories (e.g., zero-day, malware variants) using modular design and flexible schema validation.*
- ***OR2**: The ML module shall be **extensible** to include ensemble methods (e.g., Random Forest, XGBoost) and advanced neural architectures (e.g., CNNs, LSTMs) in future iterations.*
- ***OR3**: The QML/QNN layer shall support integration with alternative quantum frameworks like **Cirq**, **Braket**, or **Strawberry Fields** through standardized interfaces.*
- ***OR4**: The frontend UI shall be designed to allow **dynamic model configuration**, such as toggling hyperparameters (e.g., number of trees, kernel type) without modifying core backend code.*

5.2 Compliance and Ethical Considerations

- ***OR5**: The system shall follow **data minimization practices**, using **local storage** and **auto-deletion of uploaded datasets** after session expiration*
- ***OR6**: A disclaimer shall be included indicating that the tool is intended **solely for educational and research purposes** and should not be used in real-time production environments.*
- ***OR7**: Future versions of the system shall support **fairness analysis**, allowing users to explore potential biases in model predictions (e.g., based on IP geography or protocol).*

5.3 Documentation and Training

- ***OR8**: Developer documentation, including API references, module descriptions, and data schema formats, shall be provided in Markdown and hosted via GitHub Pages.*
- ***OR9**: A **user guide** shall be accessible from the UI, including step-by-step instructions and screenshots for dataset upload, analytics, and threat detection.*
- ***OR10**: The project shall include **training resources** such as example CSV datasets, annotated Jupyter notebooks, and test cases for demonstration.*

5.4 DevOps and CI/CD Pipeline Requirements

- ***OR11**: The project shall include a Dockerfile, docker-compose.yml, and CI scripts (e.g., GitHub Actions or GitLab CI) for automated deployment and testing.*
- ***OR12**: A staging environment shall be used for integration testing before live deployment.*
- ***OR13**: Model retraining pipelines shall be automated via script-based CLI tools with log outputs and checkpoints.*

Appendix A – Data Dictionary

Variable / Field Name	***Type***	***Description***	***Possible Values / Range***	***Operations / Requirements***
src_ip	*String*	*Source IP address from where the traffic originated*	*Valid IPv4 or IPv6 address*	*Required field, used in analytics*
dest_ip	*String*	*Destination IP address where the traffic is directed*	*Valid IPv4 or IPv6 address*	*Required field, used in analytics*
protocol	*String*	*Network protocol used*	*TCP, UDP, ICMP, etc.*	*Must be validated against known protocols*
src_port	*Integer*	*Source port number*	*0–65535*	*Optional, used in feature analysis*
dest_port	*Integer*	*Destination port number*	*0–65535*	*Optional, used in feature analysis*
timestamp	*String*	*Time at which the event was logged*	*ISO 8601 format or Unix timestamp*	*Used for time-based visualizations and analysis*
payload_size	*Integer*	*Size of the network payload in bytes*	*≥ 0*	*Used as a feature in model prediction*
flag	*String*	*TCP flag indicating connection state*	*S0, SF, REJ, RSTO, RSTR, etc.*	*Required, used in ML model and visualization*
attack_type	*String*	*Label describing whether the entry is normal or an attack*	*normal, dos, probe, r2l, u2r, etc.*	*Used as ground truth for model training/testing*
prediction	*String*	*Output label predicted by ML model*	*normal, attack*	*Generated after model evaluation*
accuracy	*Float (%)*	*Model performance metric – accuracy*	*0.0 – 100.0*	*Calculated during evaluation*
precision	*Float (%)*	*Model performance metric – precision*	*0.0 – 100.0*	*Calculated during evaluation*
recall	*Float (%)*	*Model performance metric – recall*	*0.0 – 100.0*	*Calculated during evaluation*
f1_score	*Float (%)*	*Model performance metric – F1-score*	*0.0 – 100.0*	*Calculated during evaluation*
confusion_matrix	*Matrix*	*Confusion matrix for true/false positives/negatives*	*2D Array*	*Visualized using matplotlib/seaborn*
roc_curve	*Plot Object*	*Receiver Operating Characteristic curve*	*TPR vs FPR*	*Visualized for classification performance*
csv_file_name	*String*	*Name of the uploaded file*	*Any valid file name ending in .csv*	*Required, used for session management and logging*
session_id	*String*	*Unique identifier for the current user session*	*UUID or hashed token*	*Used for tracking upload and model run*

Appendix B - Group Log

Date	Members Present	Activity	Details / Discussion Points
15-Mar-2025	All	Project Topic Finalization	Decided to build a Machine Learning-based Intrusion Detection System using network logs.
17-Mar-2025	All	Dataset Selection	Chose KDD Cup 99 dataset; discussed features like src_ip, dest_ip, flag, and attack_type.
22-Mar-2025	All	Backend Setup	Created basic Flask backend structure for file upload and model integration.
25-Mar-2025	All	Data Preprocessing	Cleaned missing values, handled outliers, encoded categorical features, and applied scaling.
29-Mar-2025	All	Model Training	Trained SVM and Random Forest models; evaluated performance. Random Forest performed best.
02-Apr-2025	All	Frontend Development	Built UI using HTML, CSS, and JavaScript for CSV upload and metric visualization.
05-Apr-2025	All	Integration	Linked frontend with backend API; ensured prediction and output visualizations worked smoothly.
09-Apr-2025	All	Testing & Debugging	Tested the system with multiple CSV files, validated results, handled exceptions.
12-Apr-2025	All	Documentation	Wrote use cases, NFRs, software architecture, and appendices.
18-Apr-2025	All	Final Review & Submission	Reviewed everything, finalized GitHub repo, cleaned UI and verified performance.